

# 需求文档和API列表

[文件不支持导出]

请根据github上的这个项目，给我们生成一个需求文档和API列表。我们四个人分别用来实现各个API

下面这份我直接按“**可落地的需求文档 + 可分工实现的 API 规格**”来写。先说明一个前提：你这条消息里没有直接贴 GitHub 链接，所以我按你上传文档里已经收敛好的路线来定范围——**正式开发基于官方 ros-navigation/navigation2，navigation2\_ignition\_gazebo\_example 只作为仿真热身参考**。这个前提和两个仓库的公开定位是一致的：前者是官方 ROS 2 导航框架，后者 README 也明确写的是 Nav2 + Gazebo 的 minimal example。□fileciteturn0file0 turn980073view5

## 一、需求文档

### 1. 项目名称

基于 Navigation2 的新手友好型单机器人导航与自动化测试系统

### 2. 项目目标

在固定地图、单机器人、Gazebo 仿真环境下，基于官方 Nav2 完成一个可演示、可测试、可扩展的课程项目。系统需要支持：

1. 按任务点序列执行导航。
2. 通过自定义区域规则影响路径规划。
3. 通过安全监控节点在异常时暂停或取消任务。
4. 批量运行测试用例并导出结果。

### 3. 范围

#### 3.1 本期必须完成

1. 固定地图。
2. 单机器人。
3. 默认 Nav2 规划与控制。
4. 你们自研的四个模块：
  - A：场景生成与自动化测试
  - B：任务状态机与任务执行
  - C：区域规则代价地图层
  - D：安全监控节点

## 3.2 明确不做

1. SLAM 建图。
2. 多机器人协同。
3. 自研全局规划器/控制器。
4. 真实机器人联调。
5. 复杂传感器融合。

这和你上传文档里的“固定地图、单机器人、默认 Nav2、自研 4 模块”的收敛方向一致。□fileciteturn0file0

## 4. 系统总流程

1. A 模块读取 `case.json`，解析出 `mission.json` 和 `zones.json`。
2. B 模块读取任务点并调用 Nav2 执行导航。
3. C 模块把区域规则写入 costmap，使路径绕开禁行区或高代价区。
4. D 模块持续订阅传感器数据，发布安全状态。
5. B 模块根据安全状态决定继续、暂停、取消或重试。
6. A 模块收集结果，输出 CSV 报告。

任务执行建议优先走 `nav2_simple_commander.BasicNavigator` 这条线，因为官方已经给了适合 Python 应用层使用的 `waitUntilNav2Active()`、`goToPose()`、`followWaypoints()`、`cancelTask()`、`isTaskComplete()`、`getFeedback()`、`getResult()` 等接口，而且这些导航调用是非阻塞的，适合你们把重试、超时和安全判断写在自己的业务循环里。turn554005view1turn554005view4

## 5. 功能需求

### FR-01 场景加载

系统必须能够从 `case.json` 加载一次实验所需的地图、任务文件、区域规则文件和运行参数。

### FR-02 任务执行

系统必须能够按顺序执行不少于 3 个 waypoint 的导航任务。

### FR-03 区域规则

系统必须支持至少 3 类区域：

1. 禁行区
2. 高代价区
3. 限速区（或减速区）

自定义区域模块做成 costmap layer 是合理路线，因为官方就提供了 Costmap2D 插件教程；插件核心运行时接口就是 `nav2_costmap_2d::Layer` 的 `onInitialize()`、`updateBounds()`、`updateCosts()` 等方法。官方也有 keepout zones 教程可直接参考。turn336779view2turn980073view3

## FR-04 安全监控

系统必须订阅 `LaserScan`，可选订阅 `Odom`，根据阈值判断当前安全状态：

1. SAFE
2. SLOW\_DOWN
3. STOP\_NOW
4. SENSOR\_FAULT

把安全模块做成独立节点也是合理路线，因为官方 Collision Monitor 本来就是一个额外安全层，直接用传感器输入做碰撞监控，而不是把安全逻辑塞进 costmap 或轨迹规划器内部。

## FR-05 安全联动

B 模块收到 `STOP_NOW` 后，必须取消当前导航任务；收到 `SAFE` 后，可以根据策略恢复或重新下发目标。

## FR-06 自动化测试

A 模块必须能批量执行至少 5 个 case，并输出 CSV 结果，内容至少包括：

1. case\_id
2. success/fail
3. elapsed\_sec
4. completed\_waypoints
5. fail\_reason
6. safety\_stop\_count

## FR-07 日志

所有模块必须输出结构化日志。推荐格式：

```
[MODULE][STATE][CODE] message
```

## FR-08 测试

每个模块至少有：

1. 单元测试
2. 一个启动级集成测试
3. 至少一个异常路径测试

## 6. 非功能需求

---

### NFR-01 环境统一

全队必须统一 Ubuntu / ROS 2 发行版，避免一半 Humble、一半 Jazzy。这个约束在你上传的规划文档里已经明确强调。  
`□fileciteturn0file0`

### NFR-02 可维护性

代码必须按包拆分，禁止四个人共写一个大包。

### NFR-03 配置驱动

阈值、路径、case 文件路径、区域规则都放在配置文件或参数里，不允许硬编码到主逻辑。

### NFR-04 鲁棒性

所有外部输入都必须有错误处理：

1. 文件不存在
2. JSON 缺字段
3. topic 超时
4. action 超时
5. scan 含 NaN/inf
6. 参数越界

### NFR-05 可演示性

系统必须能在 4 月 23 日前展示一个最小可行版本：

1. 3 个 waypoint
2. 1 个 keepout zone
3. 1 次安全停车
4. 1 张结果 CSV

---

## 二、API 设计原则

---

我建议把接口分成三层，这样四个人不会互相卡住：

1. **文件契约 API**：A 产出，B/C 消费。
2. **自定义 ROS API**：B/C/D 运行时通信。
3. **官方 Nav2 依赖 API**：B 和 C 调官方接口，不自己重复造轮子。

另外说明一下：下面的自定义接口不是 GitHub 仓库现成给你们的，而是我基于你们项目目标整理出来的“并行开发契约”。这样最适合课程开发。

---

## 三、API 列表

### 3.1 文件契约 API（MVP 必做）

#### 1) case.json

**作用：**A 模块的顶层输入，告诉系统这次跑哪个 mission、哪个 zone、超时多少。

**建议格式：**

```
{
  "case_id": "case_01",
  "map": "warehouse_map",
  "mission_file": "missions/mission_01.json",
  "zone_file": "zones/zones_01.json",
  "timeout_sec": 180,
  "repeat": 1,
  "enable_safety": true
}
```

#### 2) mission.json

**作用：**B 模块消费，定义 waypoint 序列。

```
{
  "mission_id": "mission_01",
  "frame_id": "map",
  "waypoints": [
    {"id": "P1", "x": 1.0, "y": 2.0, "yaw": 0.0},
    {"id": "P2", "x": 3.2, "y": 1.8, "yaw": 1.57},
    {"id": "P3", "x": 4.0, "y": 0.5, "yaw": 3.14}
  ]
}
```

#### 3) zones.json

**作用：**C 模块消费，定义禁行/高代价/限速区域。

```
{
  "zones": [
    {
      "zone_id": "keepout_1",
      "type": "KEEP_OUT",
      "shape": "polygon",
      "points": [[1.0, 1.0], [2.0, 1.0], [2.0, 2.0], [1.0, 2.0]],
      "cost": 254,
      "enabled": true
    }
  ]
}
```

---

## 3.2 自定义 ROS API（你们要自己实现）

---

### 1) LoadScenario.srv

服务名: /scenario/load\_case

负责人: 成员 1 (A 模块)

```
string case_file
---
bool success
string case_id
string mission_file
string zones_file
string message
```

说明:

- A 负责解析 case.json
- B 和 C 不直接自己找文件，统一通过这个服务拿路径

---

### 2) RunMission.action

动作名: /mission/run

负责人: 成员 2 (B 模块)

```
# Goal
string mission_file
uint32 max_retry_per_waypoint
bool allow_skip_waypoint

---
# Result
bool success
uint32 completed_waypoints
uint32 failed_waypoints
string final_state
string message

---
# Feedback
uint32 current_waypoint_index
string current_state
float32 distance_remaining
float32 elapsed_sec
```

说明:

- 这是 B 模块最核心的 API

- 负责把 mission 文件转成 waypoint 序列并调用 Nav2
  - 支持 cancel、retry、skip
- 

### 3) MissionState.msg

**Topic:** /mission/state

**负责人:** 成员 2 (B 模块)

```
builtin_interfaces/Time stamp
string mission_id
uint32 current_waypoint_index
uint32 total_waypoints
string state
float32 progress_percent
string detail
```

**说明:**

- 给 A 做结果采集
  - 给 D 做联动观察
  - 给 RViz/CLI 做调试显示
- 

### 4) SafetyState.msg

**Topic:** /safety/state

**负责人:** 成员 4 (D 模块)

```
builtin_interfaces/Time stamp
uint8 SAFE=0
uint8 SLOW_DOWN=1
uint8 STOP_NOW=2
uint8 SENSOR_FAULT=3
uint8 level
float32 min_range
string source
string reason
bool emergency_latched
```

**说明:**

- D 发布
  - B 订阅
  - 一旦 `level == STOP_NOW`, B 立刻 cancel 当前导航
-

## 5) ZoneRule.msg

负责人：成员 3（C 模块）

```
string zone_id
uint8 KEEP_OUT=0
uint8 HIGH_COST=1
uint8 SPEED_LIMIT=2
uint8 zone_type
string frame_id
geometry_msgs/Point[] polygon
uint8 cost
float32 speed_limit
bool enabled
```

## 6) ZoneRuleArray.msg

Topic: /zone\_layer/rules

负责人：成员 3（C 模块）

```
ZoneRule[] rules
```

说明：

- MVP 可以先不做动态 topic 更新，先让 C 直接读取 `zones.json`
- 第二阶段再把动态发布 `ZoneRuleArray` 接上

---

## 7) BatchTestResult.msg

Topic: /test/result

负责人：成员 1（A 模块）

```
string case_id
bool success
float32 elapsed_sec
uint32 completed_waypoints
uint32 safety_stop_count
string fail_reason
```

说明：

- A 负责汇总 CSV
  - 这个 topic 用于实时结果观察
-



## 8) 可选接口

### /zone\_layer/reload

类型: `ReloadZones.srv`

负责人: 成员 3

```
string zones_file
---
bool success
uint32 loaded_count
string message
```

### /safety/reset

类型: `std_srvs/Trigger`

负责人: 成员 4

用途: 清理 emergency latch, 恢复系统。

---

## 3.3 官方 Nav2 依赖 API (不要自己重写, 直接调)

你们自己的系统不是替代 Nav2, 而是**包一层应用逻辑**。B 模块应主要依赖 `BasicNavigator` 提供的应用级接口; 其背后对应的官方动作主要是 `NavigateToPose`、`NavigateThroughPoses`、`FollowWaypoints`、`ComputePathToPose`、`FollowPath`。turn554005view1turn554005view4

建议直接封装成你们自己的 `navigator_adapter.py`:

1. `wait_until_ready()`  
映射 `waitUntilNav2Active()`
2. `send_single_goal(pose)`  
映射 `goToPose()`
3. `send_waypoints(poses)`  
映射 `followWaypoints()` 或 `goThroughPoses()`
4. `poll_feedback()`  
映射 `isTaskComplete() + getFeedback()`
5. `cancel_current_task()`  
映射 `cancelTask()`
6. `get_final_result()`  
映射 `getResult()`

---

## 四、四个人的实现分工

你们最稳的做法不是“按文件随便分”, 而是按 **API 所有权**分。

## 成员 1：A 模块 nav2\_scenario\_lab

---

负责“输入与测试”。

必须实现：

1. `case.json / mission.json / zones.json` 的解析与校验
2. `/scenario/load_case`
3. `/test/result`
4. CSV 结果导出

建议代码入口：

- `scenario_manager_node.py`
- `case_loader.py`
- `mission_parser.py`
- `zone_parser.py`
- `batch_runner.py`
- `csv_reporter.py`

## 成员 2：B 模块 nav2\_mission\_manager

---

负责“任务执行与状态机”。

必须实现：

1. `/mission/run`
2. `/mission/state`
3. `navigator_adapter.py`
4. 安全联动逻辑（收到 STOP\_NOW 后 cancel）

建议代码入口：

- `mission_manager_node.py`
- `navigator_adapter.py`
- `states.py`
- `events.py`
- `retry_policy.py`
- `safety_gate.py`

## 成员 3：C 模块 nav2\_zone\_costmap\_layer

---

负责“区域规则进入 costmap”。

必须实现：

1. `ZoneRule.msg / ZoneRuleArray.msg`

2. `zones.json` 读取逻辑
3. `costmap layer` 插件主类
4. 可选 `/zone_layer/reload`

C 模块真正的代码主入口应是插件接口方法：`onInitialize()`、`updateBounds()`、`updateCosts()`、`reset()`。这些就是你们要交的“核心实现位”。turn336779view1

建议代码入口：

- `include/nav2_zone_costmap_layer/zone_layer.hpp`
- `src/zone_layer.cpp`
- `src/zone_loader.cpp`
- `src/cost_utils.cpp`
- `src/debug_polygon_publisher.cpp`

## 成员 4：D 模块 `nav2_safety_guard`

负责“传感器安全判断”。

必须实现：

1. `/safety/state`
2. `LaserScan` 过滤
3. `watchdog` 超时检测
4. 可选 `/safety/reset`

建议代码入口：

- `include/nav2_safety_guard/scan_filter.hpp`
- `include/nav2_safety_guard/safety_decider.hpp`
- `src/scan_filter.cpp`
- `src/watchdog.cpp`
- `src/safety_decider.cpp`
- `src/safety_guard_node.cpp`

这些包名和模块划分，延续了你上传文档里已经规划好的 4 模块思路。□fileciteturn0file0

## 五、MVP 必做 API 清单

为了保证 4 月 23 日能跑通，中期前只锁这 6 个：

1. `case.json`
2. `mission.json`
3. `zones.json`
4. `/scenario/load_case`

5. `/mission/run`
6. `/mission/state`
7. `/safety/state`

`ZoneRuleArray.msg`、`/zone_layer/reload`、`/safety/reset` 都可以放到第二阶段。

---

## 六、最终建议

---

你们现在最应该做的不是继续扩需求，而是**先冻结接口**。我建议本周内先把 `course_interfaces` 包建出来，把上面 5 个核心接口文件定死，然后四个人按 API 所有权分别开发。这样后面联调时，冲突会小很多。

下一步我可以直接把这份内容展开成 `course_interfaces` 的 `.msg/.srv/.action` 文件内容 + 4 个 `package` 的初始目录树。